Team 3: Presentation 1

Ronan Howard, Jingyu Huo, Yifan Xia, Taylor Massey

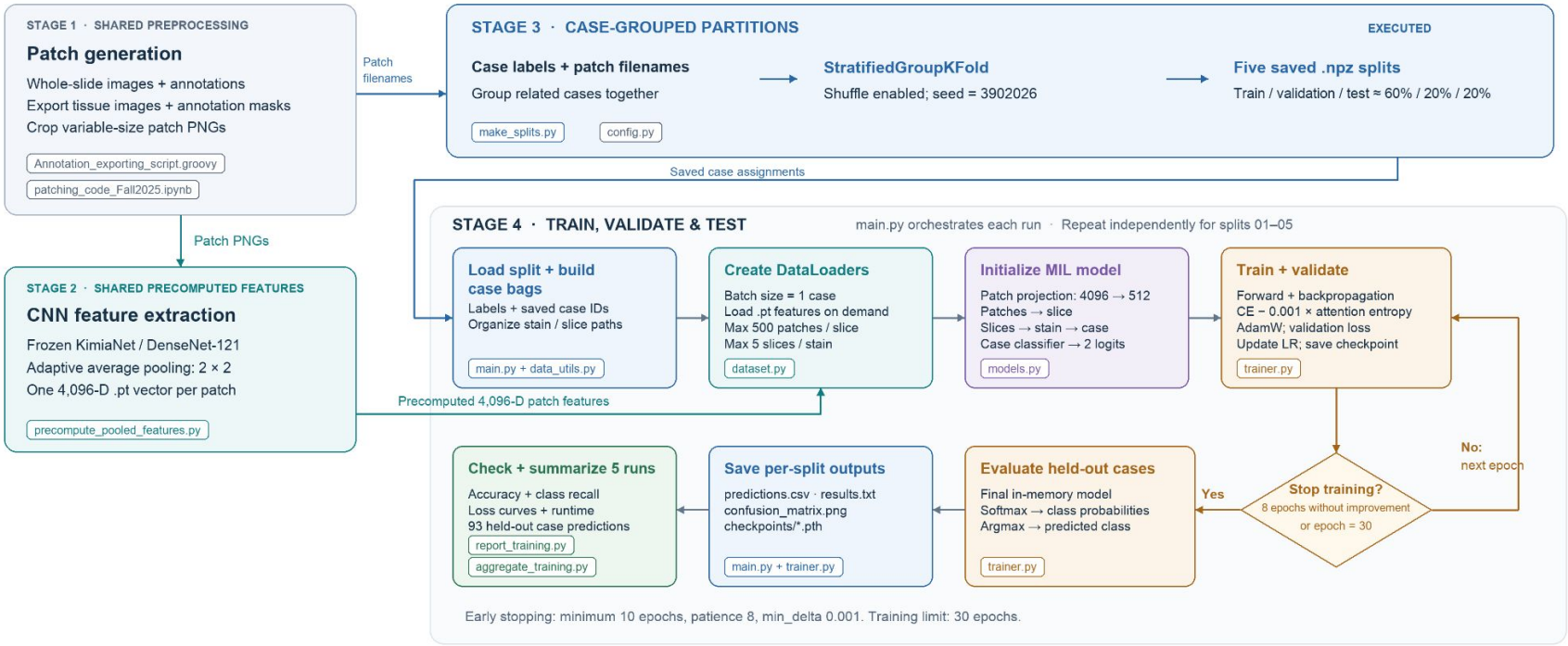
Coders

Yifan Xia & Jingyu Huo

Hierarchical Attention MIL Pipeline

From annotated slides to case-level benign / high-grade predictions

Data | Model | Training | Reporting



Execution scope	Stages 1–2 use supplied preprocessing outputs; Stages 3–4 were executed for five independent runs.
Baseline behavior	Test the final epoch in memory. best.pth is saved but is not automatically reloaded before testing.
Supporting code	config.py: settings | utils.py: seed, device and file helpers | .sbatch: Quest submission | Green boxes: added reporting utilities

Yifan

Execute Stages 3 and 4

Class Distribution Across Five Splits

93 cases: 35 benign and 58 high-grade

Split	Train			Validation			Test		
	Total	Benign	High-grade	Total	Benign	High-grade	Total	Benign	High-grade
01	57	21	36	18	7	11	18	7	11
02	56	21	35	18	7	11	19	7	12
03	55	21	34	19	7	12	19	7	12
04	55	21	34	19	7	12	19	7	12
05	56	21	35	19	7	12	18	7	11

All counts represent cases. Each split contains all 93 cases.
High-grade proportions range from 61.11% to 63.16% across all subsets.

```
seed = 3902026

sgkf = StratifiedGroupKFold(
    n_splits=5,
    shuffle=True,
    random_state=seed
)
```

```
# Data split configuration
SPLIT_CONFIG = {
    'train_ratio': 0.6,
    'val_ratio': 0.2,
    'test_ratio': 0.2,
    'stratify': True
}
```

Stage 3: Data Splitting

- Random seed: **3902026**
- StratifiedGroupKFold with shuffling enabled (High-grade proportions: 61.11%–63.16% across all subsets)
- Known related cases kept in the same subset
- Each case used for testing exactly once across five splits

Stage 4: Model Execution

- A new model trained for each split
- Identical training settings across all five runs
- Maximum 30 epochs with early stopping
- All five runs completed successfully on Quest

Yifan

/projects/e32998/STAT390_Fall2026/Team3_DivideNConquer/presentation_1/Code1_ModelTraining_23Oct_YifanXia/code/make_splits.py

Results Analysis

Test Performance Across Five Splits

Held-out test results from five completed runs

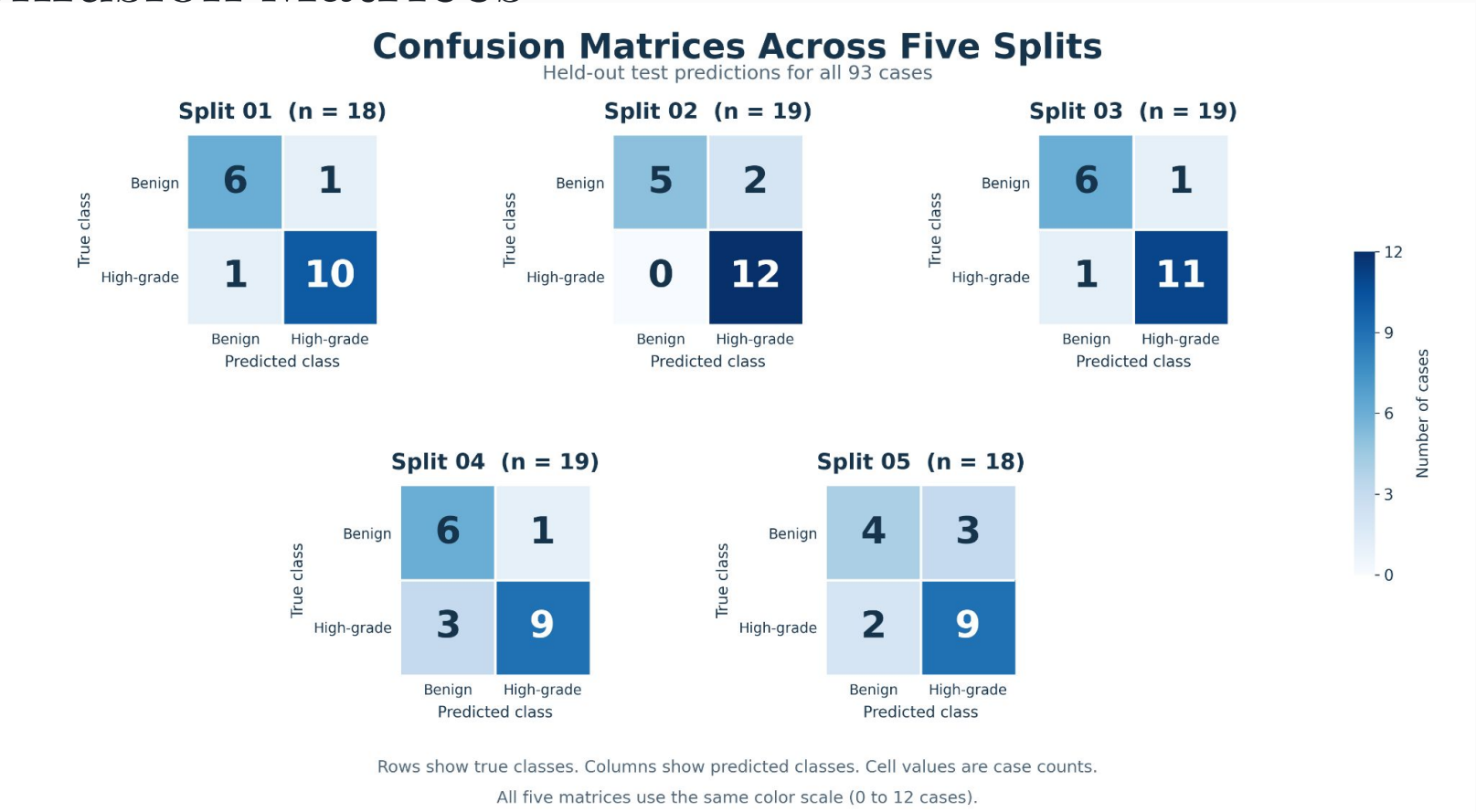
Split	Accuracy	Benign Recall	High-grade Recall	Runtime (min)
01	**88.89%**	85.71%	90.91%	8.35
02	**89.47%**	71.43%	100.00%	12.05
03	**89.47%**	85.71%	91.67%	18.70
04	**78.95%**	85.71%	75.00%	11.62
05	**72.22%**	57.14%	81.82%	13.11

Mean accuracy: 83.80% **Accuracy range: 72.22% to 89.47%**

Runtime includes data preparation, training, validation, and testing.
Queue time and report generation are excluded. Test results use the final epoch.

Yifan

Confusion Matrices



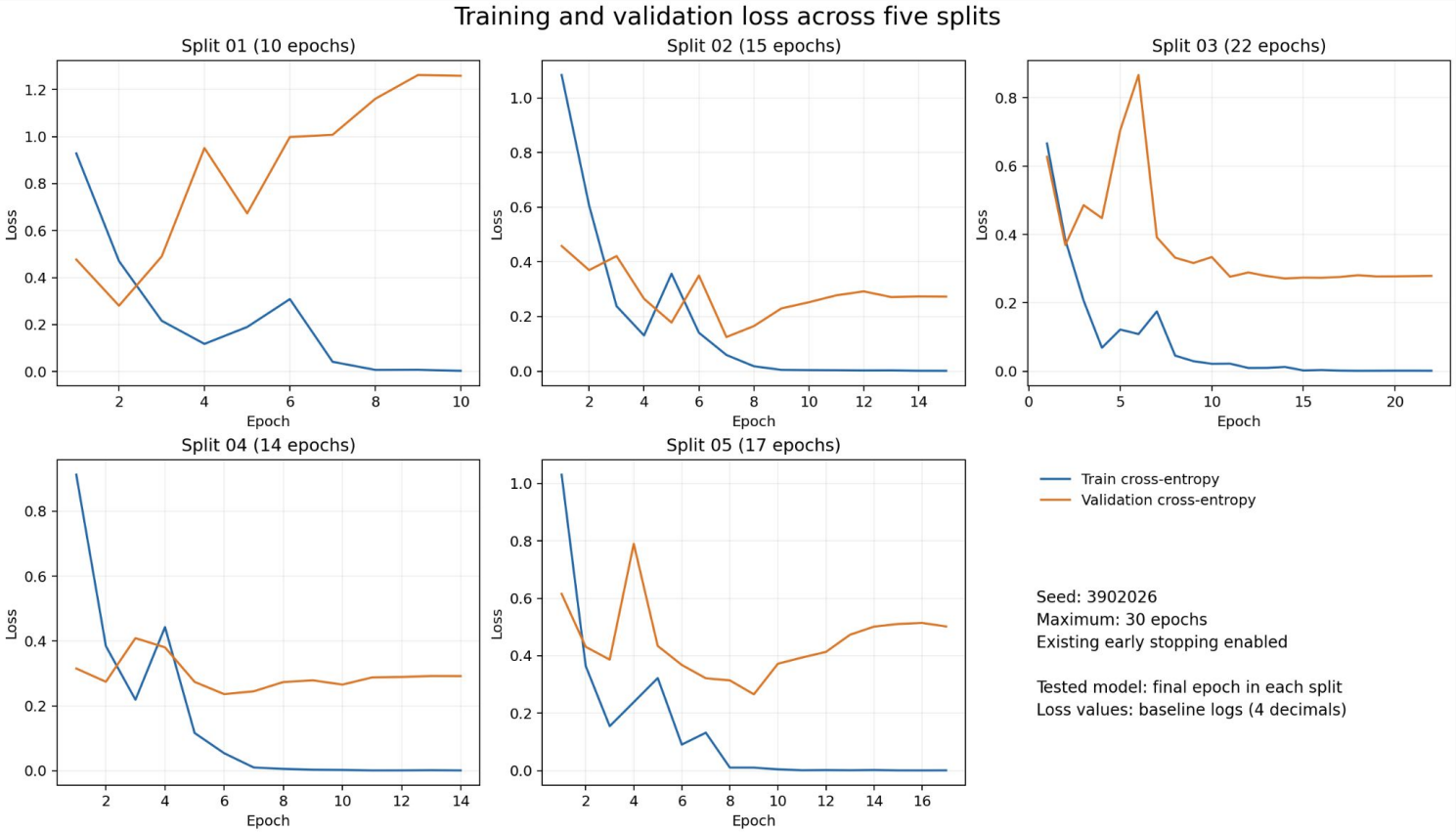
Yifan

Predicted Probability

```
split,case_id,true_label,predicted_label,correct,prob_class0,prob_class1
2,1,1,1,True,0.0006520153256133199,0.999347984790802
1,2,0,0,True,0.9998760223388672,0.0001239813573192805
4,3,1,1,True,0.0035667954944074154,0.9964331388473511
4,4,0,0,True,0.999713122844696,0.0002868653100449592
3,5,1,1,True,0.00043434472172521055,0.9995656609535217
2,7,1,1,True,8.364554604511909e-10,1.0
5,9,1,1,True,0.0006532661500386894,0.9993466734886169
3,11,1,1,True,0.4588391184806824,0.5411609411239624
2,12,1,1,True,0.28176000714302063,0.7182400226593018
4,14,1,1,True,1.8791040945487225e-10,1.0
2,15,1,1,True,0.29696598649024963,0.703033983707428
5,16,1,0,False,0.5821358561515808,0.4178641736507416
4,17,1,0,False,0.9784103035926819,0.021589744836091995
4,18,1,0,False,0.9842290878295898,0.01577090658247471
5,19,1,1,True,4.50721854576841e-05,0.9999549388885498
1,21,0,0,True,0.9857379198074341,0.014262077398598194
3,22,0,0,True,0.9999309778213501,6.901453889440745e-05
5,23,0,0,True,0.9999774694442749,2.2581421944778413e-05
4,24,0,0,True,0.9988133907318115,0.0011866411659866571
1,25,0,0,True,0.999859094619751,0.00014090753393247724
2,26,0,1,False,0.058100417256355286,0.9418995976448059
4,27,0,0,True,0.9948275685310364,0.005172400735318661
2,28,0,0,True,0.9985805749893188,0.0014194537652656436
5,30,0,0,True,0.9999934434890747,6.546521490236046e-06
4,32,1,0,False,0.993866503238678,0.0061334422789514065
1,34,0,0,True,0.9896606206893921,0.010339329019188881
1,36,0,0,True,0.9899311065673828,0.010068871080875397
4,38,1,1,True,1.9527024619492295e-07,0.9999997615814209
5,40,1,1,True,3.3941676065296633e-06,0.9999966621398926
3,41,1,1,True,2.1696013874361597e-08,1.0
1,42,1,1,True,0.00234282691963017,0.9976571798324585
2,43,0,0,True,0.9996504783630371,0.0003494739066809416
2,44,1,1,True,0.0033552737440913916,0.9966447353363037
5,45,0,1,False,5.717685053241439e-05,0.9999427795410156
2,46,0,1,False,0.04497889056801796,0.95502108335495
2,47,1,1,True,0.0009272568277083337,0.9990726709365845
3,48,1,1,True,2.6376199457445182e-05,0.9999736547470093
2,49,1,1,True,0.0001714894315227866 2,0.9998284578323364
1,50,1,1,True,0.028918607160449028,0.9710813760757446
1,51,1,1,True,0.0006422455771826208,0.9993577599525452
4,52,1,1,True,1.4920428839104716e-05,0.9999850988388062
5,53,0,0,True,0.9881530404090881,0.011846990324556828
3,54,1,1,True,0.0016002327902242541,0.9983997941017151
4,55,1,1,True,1.2390620751645542e-09,1.0
3,56,0,1,False,0.4014878571033478,0.5985121726989746
5,57,1,1,True,4.5476986997528e-05,0.9999544620513916
3,58,1,1,True,1.1932048664675676e-06,0.9999988079071045
5,59,1,1,True,2.942882701972849e-07,0.9999997615814209
3,60,1,1,True,0.00041903226519934833,0.9995809197425842
1,61,1,1,True,0.03853901848196983,0.9614610075950623
3,63,1,1,True,7.650519107116338e-10,1.0
2,64,1,1,True,0.006174113135784864,0.9938259124755859
4,66,1,1,True,8.430826170524597e-08,0.9999998807907104
1,67,1,1,True,0.03926704078912735,0.9607329368591309
1,68,1,1,True,0.0017208943609148264,0.9982790946960449
5,69,1,0,False,0.9521954655647278,0.04780452325940132
1,70,1,1,True,0.005615246947854757,0.994384765625
2,72,1,1,True,0.04201987758278847,0.9579800367355347
4,73,1,1,True,0.0007058191113173962,0.9992941617965698
5,75,1,1,True,8.036075996642467e-06,0.9999920129776001
3,77,0,0,True,0.9991225600242615,0.000877481244970113
1,78,0,1,False,0.06729933619499207,0.9327006340026855
3,79,0,0,True,0.9830586910247803,0.01694127358496189
4,80,1,1,True,3.5190265634810203e-07,0.9999996423721313
3,82,0,0,True,0.9996782541275024,0.0003217692137695849
4,83,1,1,True,1.9984300081432593e-07,0.9999997615814209
1,84,1,0,False,0.978742241859436,0.0212577972561121
4,85,0,0,True,0.9612633585929871,0.03873661160469055
5,86,0,1,False,0.002793405437842011 5,0.9972066283226013
5,87,1,1,True,1.977167084987741e-05,0.9999802112579346
3,88,1,1,True,0.0020092478953301907,0.9979907274246216
1,89,1,1,True,0.014588499441742897,0.9854114651679993
4,90,0,0,True,0.9999969005584717,3.1383024179376 66e-06
1,91,1,1,True,0.006930218078196049,0.9930697083473206
3,92,1,1,True,0.3964846730232239,0.6035153269767761
5,93,0,0,True,0.9945461750030518,0.005453864112496376
5,94,0,1,False,0.0008430422749370337,0.9991569519042969
5,95,1,1,True,0.010247469879686832,0.9897525310516357
4,96,0,1,False,0.0022789877839386463,0.9977210164070129
3,97,1,0,False,0.9867987036705017,0.013201242312788963
3,98,1,1,True,0.0008009131415747106,0.9991990923881531
2,99,1,1,True,6.165166269056499e-05,0.9999383687973022
2,100,1,1,True,1.6373176549677737e-05,0.9999836683273315
1,101,1,1,True,1.1899946912308224e-06,0.9999988079071045
5,104,1,1,True,8.180134500435088e-06,0.999991774559021
2,106,1,1,True,2.1232619474176317e-05,0.99997878074646
3,107,0,0,True,0.9999411106109619,5.8870904467767105e-05
3,108,0,0,True,0.9963405728340149,0.0036593717522919 18
2,109,0,0,True,0.850398600101471,0.14960141479969025
2,110,0,0,True,0.9999067783355713,9.317870717495 68e-05
2,111,0,0,True,0.9991705417633057,0.0008295212755911052
4,118,0,0,True,0.9992266893386841,0.0007732577505521476
1,119,0,0,True,0.999402642250061,0.0005973094375804067
```

Yifan

Train and Validation Loss Curves



Yifan

Critical reflections

Test evaluation uses the final model rather than the best checkpoint

- Training saves `best.pth`, but testing uses the final epoch's weights.

```
Epoch 5/30 (LR: 2.08e-04)
Train Loss: 0.1195 | Train CE: 0.1203 | Train Patch Entropy: 0.8734 | Val Loss: 0.2279 | Val Acc: 0.8421
Epoch Time: 135.80s
New best validation loss: 0.2279
```

```
Epoch 13/30 (LR: 4.45e-05)
Train Loss: 0.0004 | Train CE: 0.0012 | Train Patch Entropy: 0.7997 | Val Loss: 0.5910 | Val Acc: 0.8421
Epoch Time: 108.69s
No improvement for 8 epoch(s)
Checkpoint saved: /home/cny2467/MIL_training/final_runs/run_20261022_015558/checkpoints/20261022_022713_Hierar

Early stopping triggered at epoch 13. Best val loss: 0.2279

Training completed!

============================================================
EVALUATING MODEL
============================================================

Test Results:
  Test Loss: 0.3368
  Test Acc:  0.8889
  Samples:   18
```

Jingyu

Critical reflections

```python
class MILTrainer:
    def train(
            # Check best / early stop
            is_best = False
            if val_loss < (self.best_val_loss - self.early_stopping_min_delta):
                self.best_val_loss = val_loss
                self.epochs_without_improvement = 0
                is_best = True
                print(f"New best validation loss: {val_loss:.4f}")
            else:
                self.epochs_without_improvement += 1
                print(f"No improvement for {self.epochs_without_improvement} epoch(s)")

            # Save checkpoint periodically
            if save_every > 0 and ((epoch + 1) % save_every == 0):
                self.save_checkpoint(epoch + 1, arch=arch, is_best=is_best)

            # Early stopping
            if self.use_early_stopping:
                if (epoch + 1) >= self.early_stopping_min_epochs and self.epochs_without_improvement >= self.early_
                    print(f"\nEarly stopping triggered at epoch {epoch + 1}. Best val loss: {self.best_val_loss:.4f
                    break

        print("\nTraining completed!")
        self.save_loss_curve(os.path.dirname(self.checkpoint_dir) if self.checkpoint_dir else ".")
```

```python
if not args.eval_only:
    # Train the model
    print("\n" + "=" * 60)
    print("TRAINING MODEL")
    print("=" * 60)
    trainer.train(
        train_loader=train_loader,
        val_loader=val_loader,
        epochs=args.epochs,
        start_epoch=start_epoch
    )

# Evaluate on test set
print("\n" + "=" * 60)
print("EVALUATING MODEL")
print("=" * 60)
test_results = trainer.evaluate(
    test_loader,
    save_predictions=True,
    output_dir=run_dir,
    checkpoint_name=args.resume if args.resume else None
)
```

Suggestion:

call `load_checkpoint("checkpoints/best.pth")` at the start of `evaluate()`

Jingyu

Critical reflections

Repeated loading of patch embeddings

- `_load_slice_tensor` function in `dataset.py` file calls `torch.load` for every patch on every epoch.

```python
def __getitem__(self, idx: int) -> Dict[str, Any]:
    case_id, stain_map = self.items[idx]
    label = torch.tensor(self.label_map[case_id], dtype=torch.long)

    stain_slices: Dict[str, List[torch.Tensor]] = {}
    for stain in self.stains:
        slice_lists = stain_map.get(stain, []) or []

        if self.max_slices_per_stain is not None and len(slice_lists) > self.max_slices_per_stain:
            slice_lists = slice_lists[:self.max_slices_per_stain]

        tensors_for_stain: List[torch.Tensor] = []
        for sl in slice_lists:
            if not sl:
                continue
            sl_tensor = self._load_slice_tensor(sl)

            if sl_tensor is None:
                if self.drop_empty_slices:
                    # skip empty slice instead of crashing
                    continue
                raise RuntimeError(f"Empty slice encountered in case {case_id}, stain {stain}")

            tensors_for_stain.append(sl_tensor)

        stain_slices[stain] = tensors_for_stain

    return {"case_id": case_id, "stain_slices": stain_slices, "label": label}
```

```python
def _load_slice_tensor(self, patch_paths: List[str]) -> Optional[torch.Tensor]:
    patch_paths = list[str](patch_paths)
    if self.shuffle_patches:
        random.shuffle(patch_paths)

    if self.per_slice_cap and len(patch_paths) > self.per_slice_cap:
        patch_paths = patch_paths[:self.per_slice_cap]

    vecs = []
    for p in patch_paths:
        fp = self._patch_path_to_feat_path(p)
        try:
            v = torch.load(fp, map_location="cpu")  # (4096,)
            if v is None:
                continue
```

Suggestion:

Keep the tensors in a `{filepath: tensor}` dictionary, about 1.3 GB. Later epochs look up the tensor instead of calling `torch.load`.

Jingyu

Critical reflections

Different splitting path.

Without `--load_splits`, training uses `split_by_case_stratified`, which ignores `GROUPED_CASES`

Group-aware splitting:

```python
# Identify reasonable split ratios
ratios = [train_ratio, val_ratio, test_ratio]
denom = simplify_split_ratios(ratios, max_ratio_den)

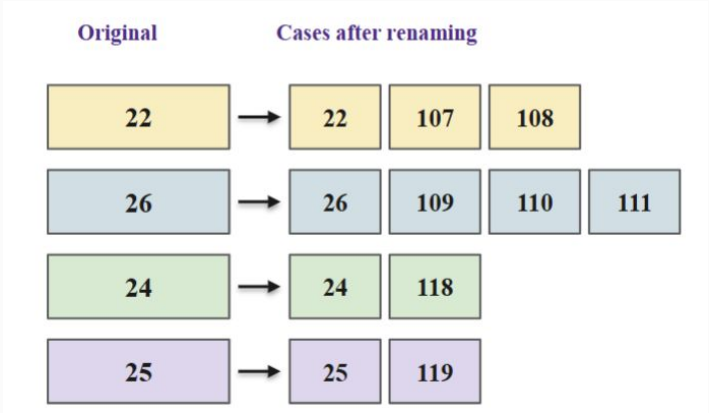
# Setup groups so that pseudo-cases can be mapped back to original case
# ex: group: (22, 107, 108), these numbers inside grousp will be replaced by 22
groups = np.array(case_ids)
for group in grouped_cases:
    mask = np.isin(groups, group)
    np.place(groups, mask, group[0])

# Perform split according to smallest unit that can represent one split set
sgkf = StratifiedGroupKFold(n_splits=sum(denom)) # shuffle disabled for causing deviations of labels ratio
sgkf_splits = sgkf.split(
    np.array(case_ids),
    np.array([case_to_label[case_ids] for case_ids in case_ids]),
    groups # case with same group id will be in the same set(train/test) for each of five splits
)

# Folds of only "test" for non overlapping sets
splits = []
for _, test_index in sgkf_splits:
    splits.append([case_ids[index] for index in test_index])

return case_to_label, denom, splits
```

GROUPED_CASES from config — e.g. (22, 107, 108) — are mapped to one group id so related benign cases never straddle splits



Jingyu

Critical reflections

Two path ->

Suggestion:

put the group-aware splitting logic in one shared function and use it wherever splits are created. Alternatively, the program could require a precomputed split file.

ordinary stratified train_test_split calls without accounting for the case groups ->

Jingyu

```python
def prepare_data(args):
    if args.load_splits:
        # Load existing splits
        print(f"Loading existing splits from: {args.load_splits}")
        splits_data = load_data_splits(args.load_splits)
        train_cases_set = set[Any](splits_data['train_cases'])
        val_cases_set = set[Any](splits_data['val_cases'])
        test_cases_set = set[Any](splits_data['test_cases'])

        # Map loaded case IDs back to slices
        train_slices = [(case_id, slice_id) for (case_id, slice_id) in slice_to_class.keys() if case_id in train_c
        val_slices = [(case_id, slice_id) for (case_id, slice_id) in slice_to_class.keys() if case_id in val_cases
        test_slices = [(case_id, slice_id) for (case_id, slice_id) in slice_to_class.keys() if case_id in test_cas

        print(f"Loaded splits - Train: {len(train_slices)}, Val: {len(val_slices)}, Test: {len(test_slices)}")
    else:
        # Split data by case (stratified)
        train_slices, val_slices, test_slices = split_by_case_stratified(
            slices_by_class, random_state=args.seed
        )
```

```python
122  def split_by_case_stratified(slices_by_class: Dict, random_state: int = 42) -> Tuple[List, List, List]:
147      case_train, case_temp, y_train, y_temp = train_test_split(
148          case_ids,
149          case_labels,
150          test_size=(1 - train_ratio),
151          stratify=case_labels,
152          random_state=random_state,
153      )
154
155      # Second split: val vs test from temp
156      val_size = val_ratio / (val_ratio + SPLIT_CONFIG["test_ratio"])
157      case_val, case_test, _, _ = train_test_split(
158          case_temp,
159          y_temp,
160          test_size=(1 - val_size),
161          stratify=y_temp,
162          random_state=random_state,
163      )
```

Mathlete

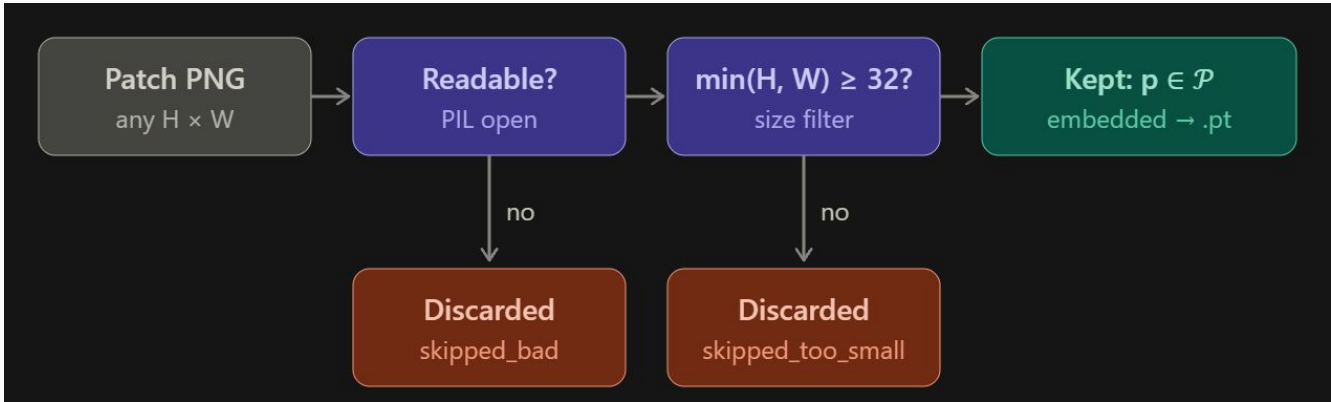
Data Preprocessing

- Patch filter: keep patch p only if readable and $\min(w_□, h_□) \geq 32$ px
  Discarded fraction = skipped_too_small / total_png = 1,825 / 83,630 = 2.18%

- Standardisation (per RGB channel, ImageNet constants): $\tilde{x} = (x/255 - \mu) / \sigma$
  $\mu = (0.485, 0.456, 0.406)$, $\sigma = (0.229, 0.224, 0.225)$ → frozen KimiaNet → AvgPool 2×2 → $z_□ \in \mathbb{R}^{4096}$

- Hierarchy: case $c \in C$ ($|C| = 93$) → stains {H&E, Melan-A, SOX10} → first K = 5 slices per stain → M = 500 patches per slice
  Bag: $B_c = \{(s, \ell, p)\}$, $|B_c| \leq 3 * 5 * 500 = 7{,}500$

- Groups: g(c) = parent patient {22, 107, 108} → 22 {24, 118} → 24 {25, 119} → 25 {26, 109, 110, 111} → 26 else g(c) = c
  Constraint: All pseudo-cases from the same patient must land in the same fold, so no same patient lands across train, validation and test.



Ronan

Splits

- Each fold: 7 benign / 11–12 high-grade
- Split r rotates the folds:
  test = F_r, val = F_{r-1} ($F_0 = F_5$),
  train = remaining 3 folds
- Train: 55–57 cases (21 B / 34–36 H)
  Val, Test: 18–19 cases (7 B / 11–12 H)
- Benign share 0.37–0.39 in every set
- Every case is a test case exactly once

split file	F1 7B / 11H	F2 7B / 12H	F3 7B / 12H	F4 7B / 12H	F5 7B / 11H	train n (B / H)
split 01	test	train	train	train	val	57 (21 / 36)
split 02	val	test	train	train	train	56 (21 / 35)
split 03	train	val	test	train	train	55 (21 / 34)
split 04	train	train	val	test	train	55 (21 / 34)
split 05	train	train	train	val	test	56 (21 / 35)

Ronan

Tensor Transformations

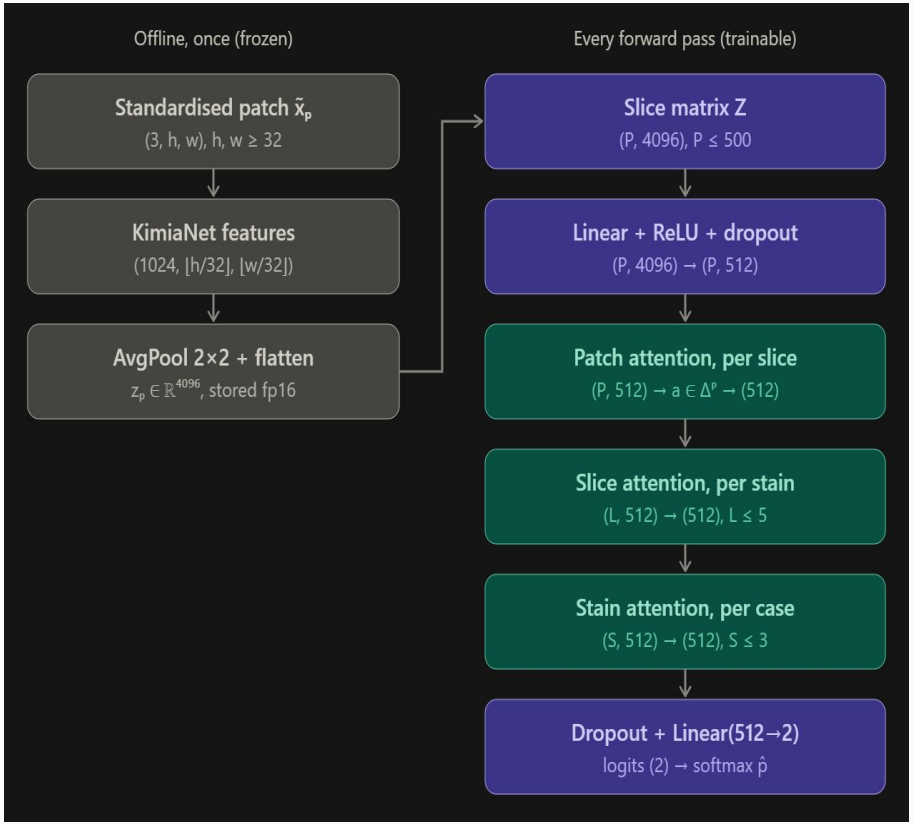
- Offline: (3, h, w) → KimiaNet (1024, h/32, w/32) → AvgPool 2×2 → (4096)

- Per case: slice (P, 4096) → Linear + ReLU + Dropout (P, 512)
  → patch attention (512) → slice attention over L ≤ 5 (512)
  → stain attention over S ≤ 3 (512) → Dropout + Linear (2)
  → softmax

- Gated attention:
  $e_i = w^T [\tanh(V h_i) \odot \text{sigmoid}(U h_i)]$, $a_i = \text{softmax}(e)_i$,
  $\text{pool} = \Sigma_i a_i h_i$ $V, U \in \mathbb{R}^{65,536}$

- Dropout (training only): $\tilde{u} = m \odot u / 0.74$,
  $m \sim \text{Bernoulli}(0.74)$

Ronan



Parameters

Component	Params	Status
KimiaNet encoder (DenseNet-121)	6,953,856	Frozen
Patch projector	2,097,664	Estimated
Attention pools	394,371	Estimated
Classifier	1,026	Estimated
Total	**9,446,917**	**2,493,061 estimated (26.4%)**

Tuned (Optuna)	Value	Hard-coded	Value
Learning rate	2.08e-4	Embed / Attention Dim	512 / 128
Weight decay	3.05e-4	Patch cap / slice cap	500 / 5
Benign class weight	2.5285	Epochs / batch size	30 / 1 case
Dropout (2 layers)	0.2599	LR plateau	0.2139, patience 4
Entropy λ	0.001	Early stopping	patience 8, min 10 epochs
		Min patch size, threshold	32 px, 0.5

Loss Function

- Per-case training loss: $\mathcal{L}_c = CE_c - \lambda \cdot H_c$
- Weighted cross-entropy: $CE_c = w_y(-\log \hat{p}_y) / w_y = -\log \hat{p}_y$
  $w = (2.5285, 1.0)$; mean reduction with batch = 1 so the class weight cancels (discussed later)
- Patch-attention entropy: $H_c = (1/N_c) \Sigma_slices [-\Sigma_i a_i \log a_i / \log P] \in [0, 1]$, $\lambda = 0.001$
- Validation loss: $-\log \hat{p}_y$ only (no entropy or dropout)

Full Loss:

$$\mathcal{L}(\theta) = (1 / |C_train|) \cdot \Sigma_c [- w(y_c) \cdot \log \hat{p}(y_c \mid \theta) / w(y_c) - \lambda \cdot (1 / N(c)) \cdot \Sigma_(s,\ell) (- \Sigma_i a_i \cdot \log a_i) / \log P(s,\ell)]$$

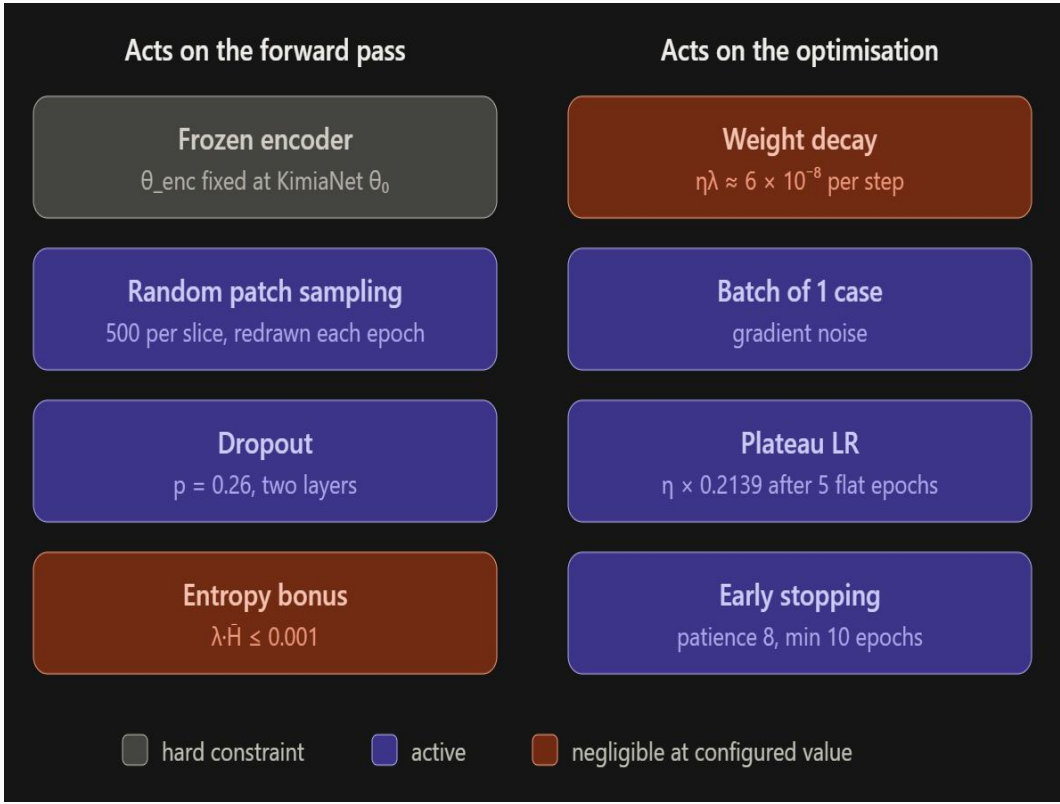
Effective per-case loss:

$$\mathcal{L}(c) = -\log \hat{p}(y_c) - 0.001 \cdot H(c), H(c) \in [0, 1]$$

Ronan

Regularisers

1. Frozen encoder: $\theta_enc \equiv \theta_0$ (KimiaNet), no gradient
2. Random patch sampling: first 500 of a random permutation, redrawn every epoch (train only)
3. Dropout: $\tilde{u} = m \odot u / (1 - p)$, $p = 0.26$, (train only)
4. Entropy bonus: $\mathcal{L} \leftarrow \mathcal{L} - \lambda H$, $\lambda = 0.001$
5. Weight decay (AdamW): $\theta = (1 - \eta\lambda_wd)\,\theta - \eta\,\hat{m} / (\sqrt{\hat{v}} + \varepsilon)$, $\lambda_wd = 3.05e\text{-}4$
6. Batch of 1 case: $g = \nabla\mathcal{L}_c(\theta)$, random case order each epoch
7. Plateau LR: $\eta \leftarrow \max(0.2139\,\eta, 1e\text{-}6)$ after > 4 epochs without val improvement
8. Early stopping: stop if epoch ≥ 10 and 8 epochs without val loss falling below best − 0.001



Reflection

Not flawless.

The class weight cancels at batch = 1. Since we use batch_size = 1 and case_data = batch[0], the loss is computed one case at a time:

PyTorch's mean reduction: CE = Σ w(y□)·(−log $\hat{p}_n$) / Σ w(y□) with only one case gives: w(y)·(−log $\hat{p}$) / w(y) = (−log $\hat{p}$) since we are summing over the one batch

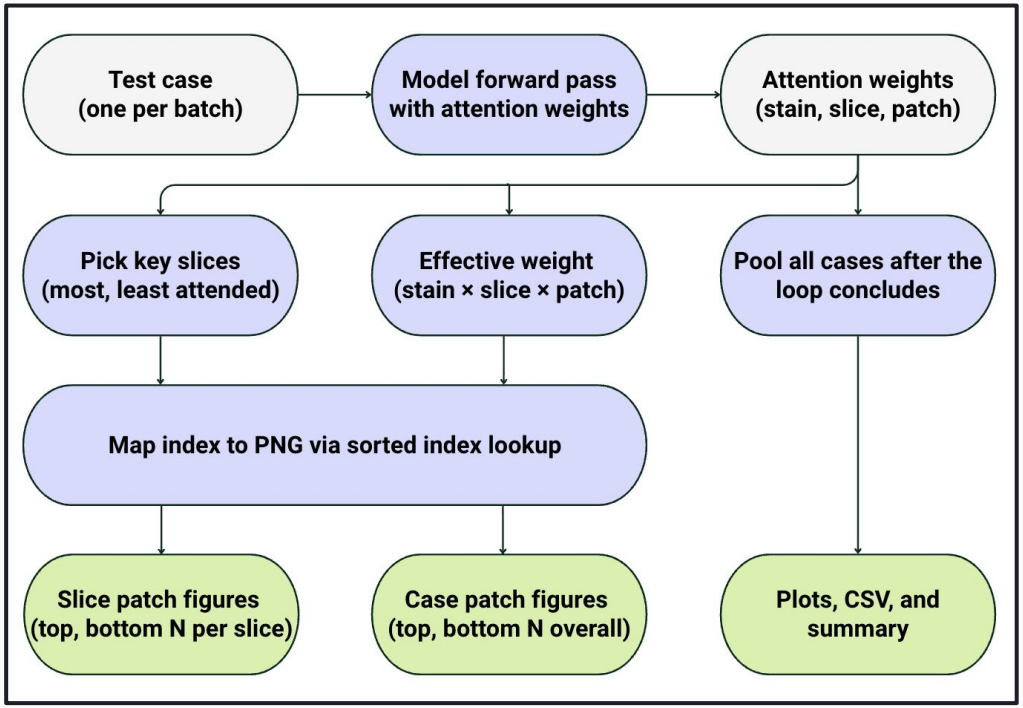
The benign weight of 2.5285 never actually affects the loss, so the benign cases are never up-weighted and Optuna tuned a parameter with no effect

Fix: loss = w(y) · F.cross_entropy(logits, y) in trainer.py so the class weight is not cancelled in mean reduction and retune the benign weight

Ronan

Analyst

attention_analysis.py

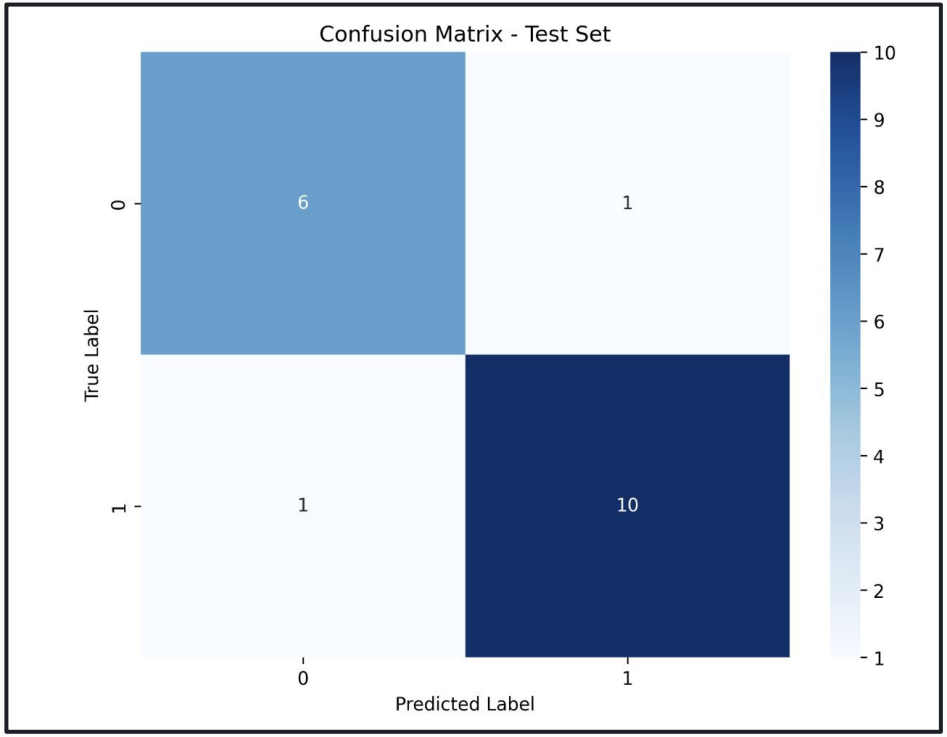


- Each test case goes through the model once, returning attention at three levels: patch, slice, and stain
- Because attention weights at each level normalize to 1 via softmax, the product across all patches in the case sums to 1
- Top and bottom patches are mapped back to PNG files by position

/projects/e32998/STAT390_Fall2026/Team3_DivideNConquer/presentation_1/taylor_code/attention_analysis.py

Case Selection

- Fold 5 test set containing 18 cases (7 benign, 11 high-grade)
- Model demonstrates high overall accuracy, but has two important edge-case failures to note:
  - **False Negative:** Case 18
  - **False Positive:** Case 78



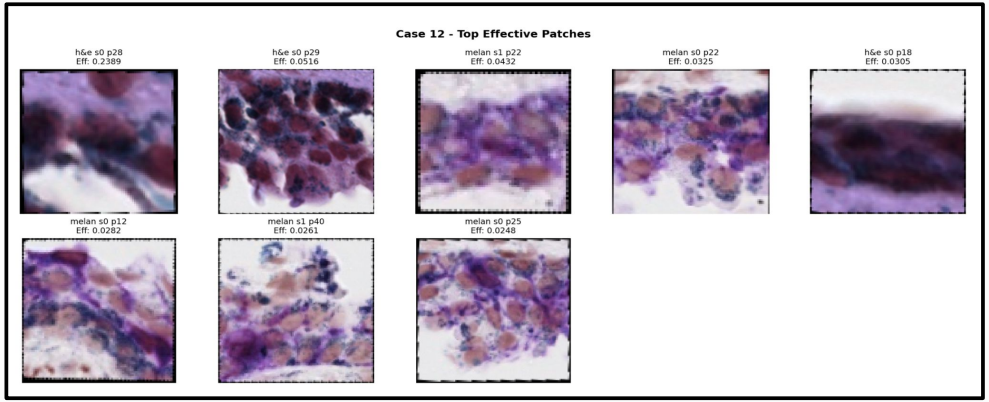
Taylor

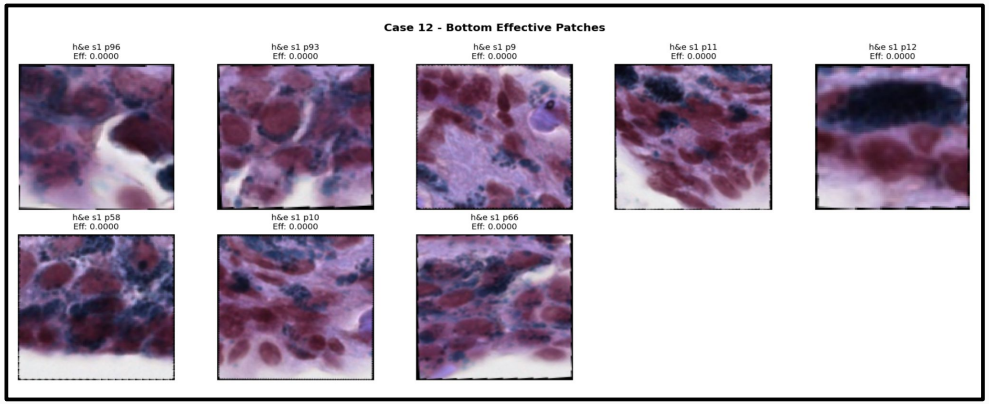
Case 12: True Positive

- P(Benign): 0.0035
- P(High-Grade): 0.9965

Observations

- Top effective patches hold ~48% of attention
  - Show crowded cells with dark nuclei
- Bottom patches come from H&E Slice 1, which the model de-prioritized at the slice-pooling level

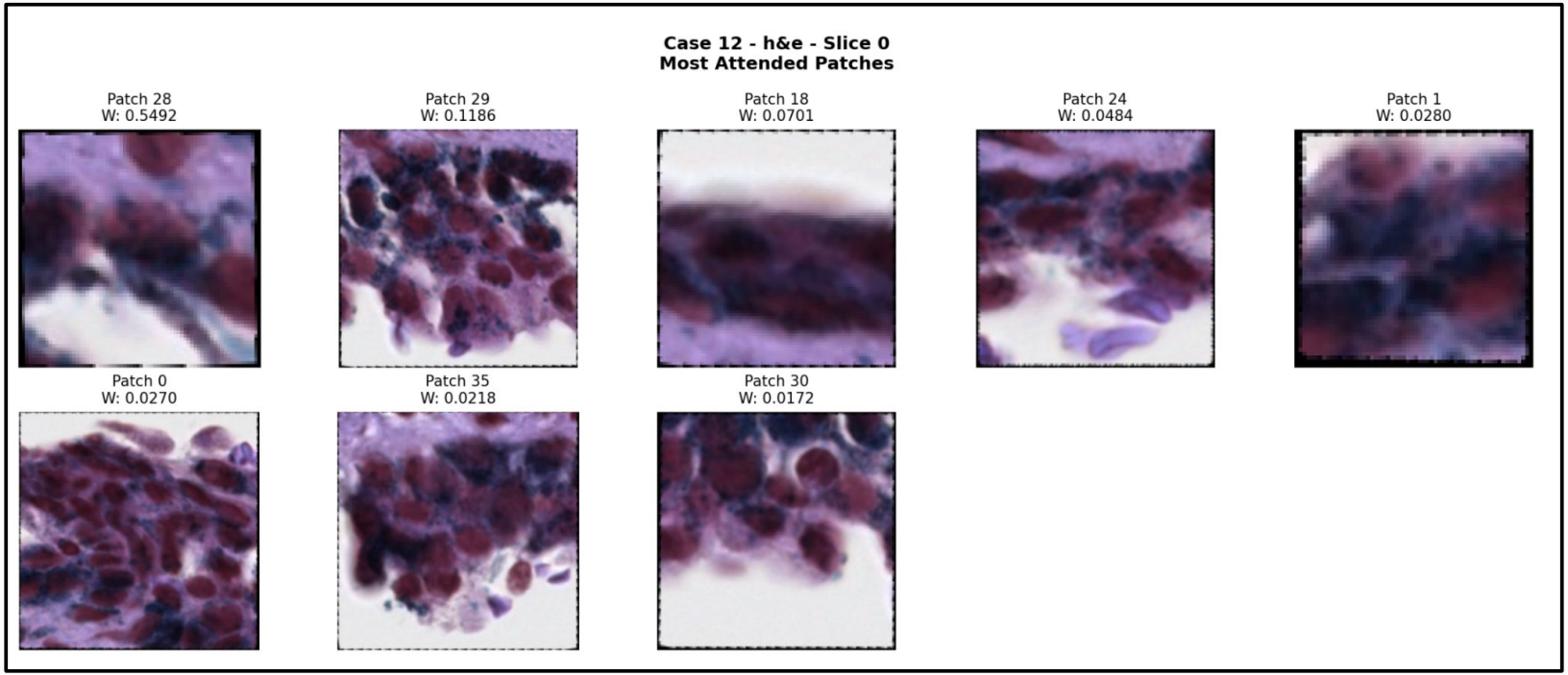




Taylor

Case 12: True Positive

Note: Only one slice is shown due to presentation time constraints, but all can be found in the Appendix



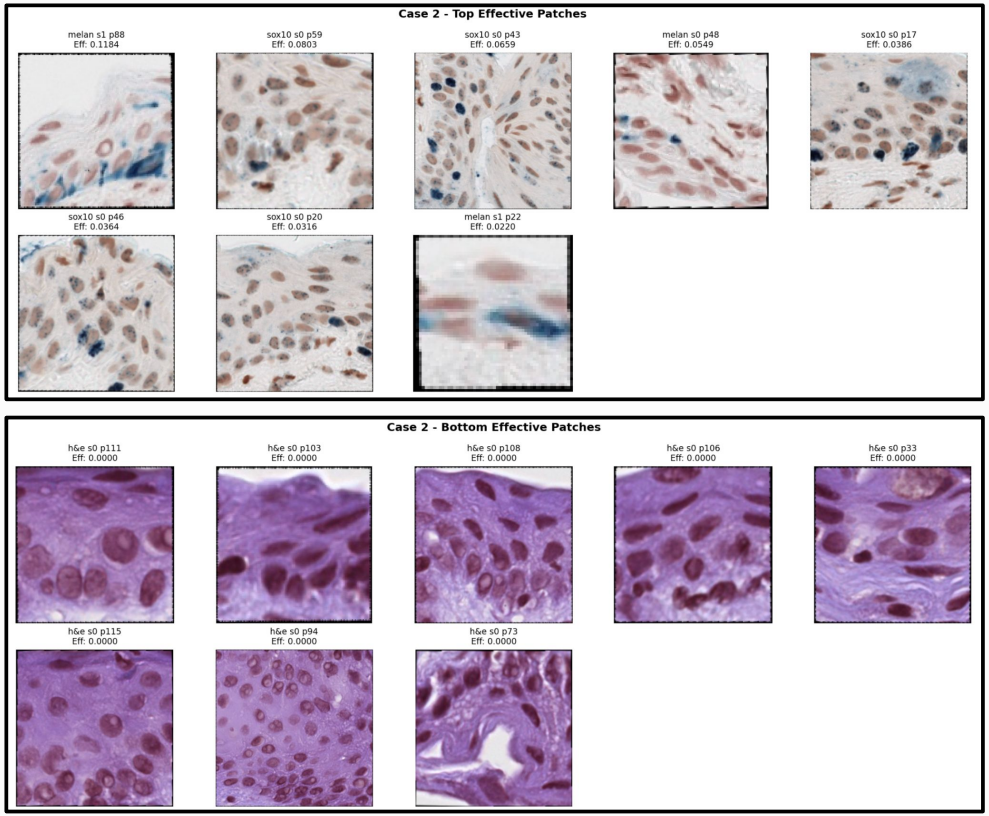
Taylor

Case 2: True Negative

- P(Benign): 0.9997
- P(High-Grade): 0.0003

Observations

- Top patches are all Melan-A and SOX10
  - Pale tissue with evenly spaced, oval nuclei
- All bottom patches are from one H&E slice with essentially zero weight



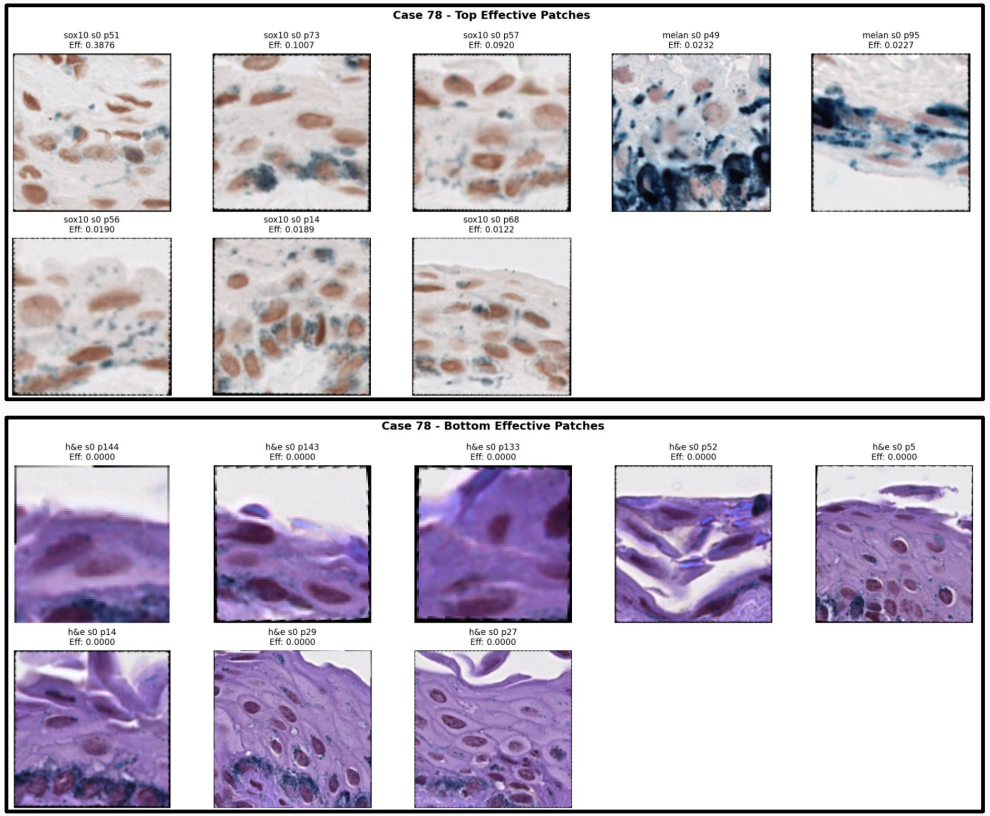
Taylor

Case 78: False Positive

- P(Benign): 0.2413
- P(High-Grade): 0.7587

Observations

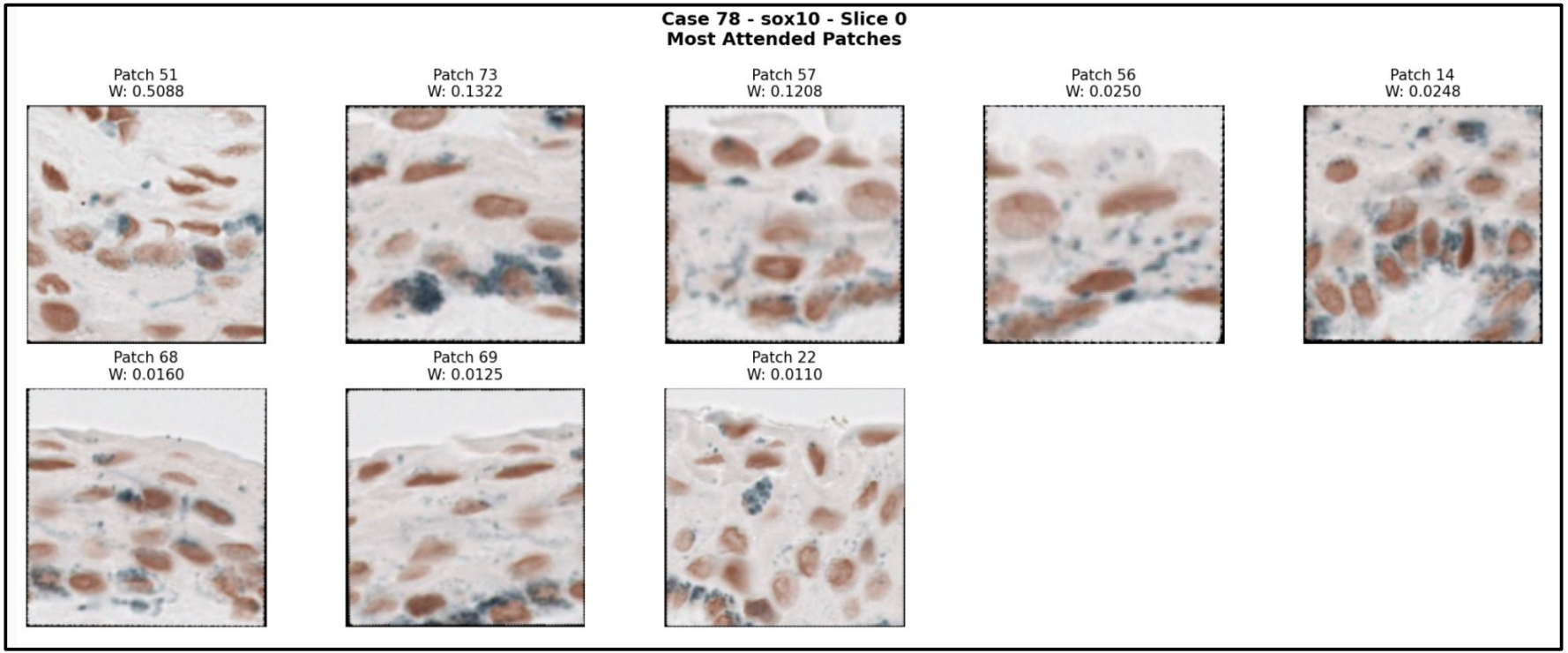
- One SOX10 patch has 39% of the case's attention (most concentrated of the four)
- SOX10 and Melan-A patches show dense, dark pigmentation that mimics malignant cellular crowding



Taylor

Case 78: False Positive

Note: Only one slice is shown due to presentation time constraints, but all can be found in the Appendix



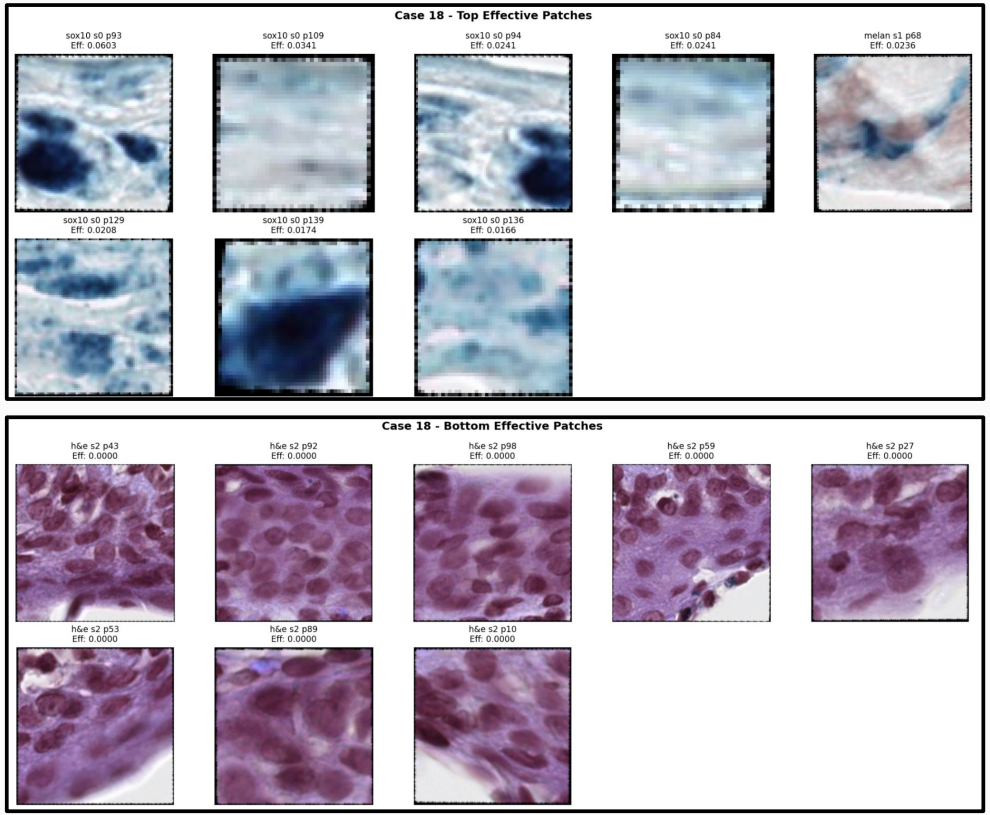
Taylor

Case 18: False Negative

- P(Benign): 0.9764
- P(High-Grade): 0.0236

Observations

- Attention is spread out, with the highest effective weight reaching only ~0.06 on SOX10 Slice 0
- Several top patches look pale and blurry with little visible detail, and others show dark blue areas



Taylor

Reflection

Successes

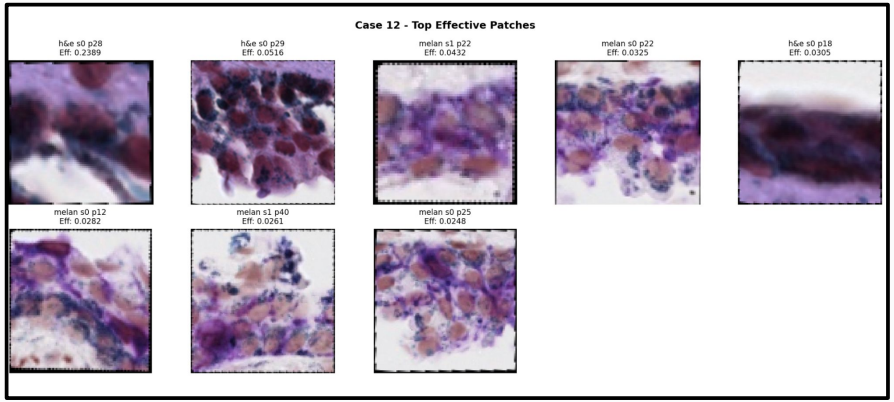
- The pipeline ranks patches at three levels and saves labeled figures with attention values for all test cases
  - Effective weights sum to 1 within a case, so they are read as shares
- Attention looked more concentrated in the false positive (top patch, 39%) rather than the false negative (6%)

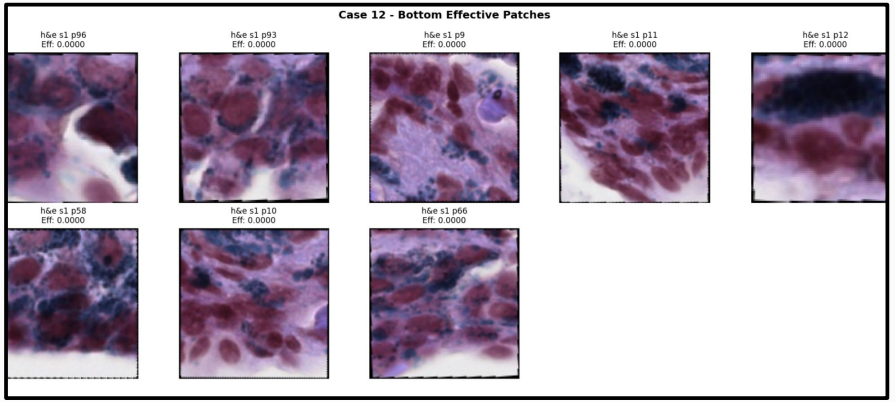
Problems

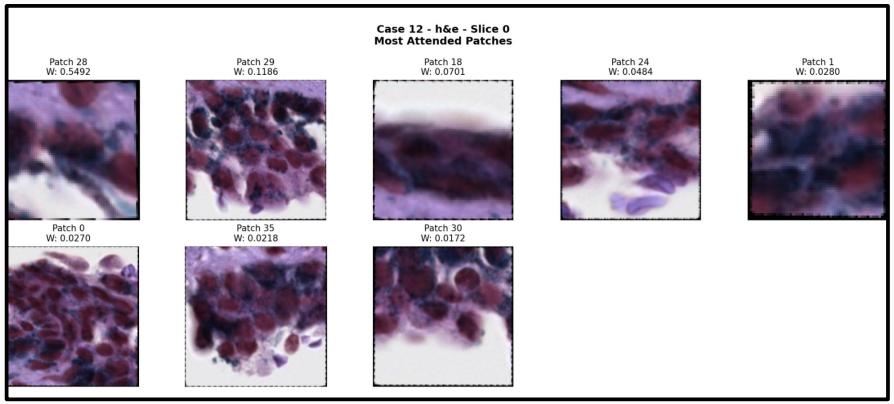
- Potential indexing issues
  - Model lists patches in text order, but figure code lists them in numerical order
  - **Fix:** Sort both the same way or have the dataset save exact file list for figure code to read
  - Extra, missing, or unreadable files shift positions (folders have 83,630 PNGs but only 81,805 embeddings)
  - **Fix:** Build both from one shared list
- High attention does not guarantee a patch drove the prediction
  - **Potential test:** Remove the top patches and see if a prediction changes

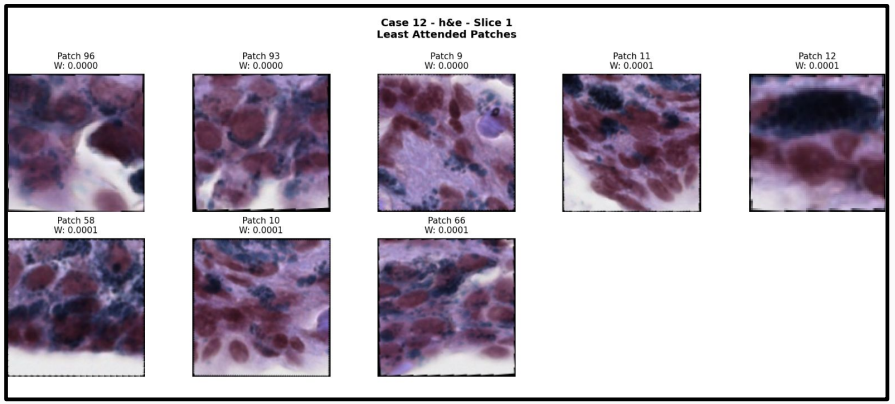
Appendix

Appendix: Full Stain & Slice Visualizations (Case 12)



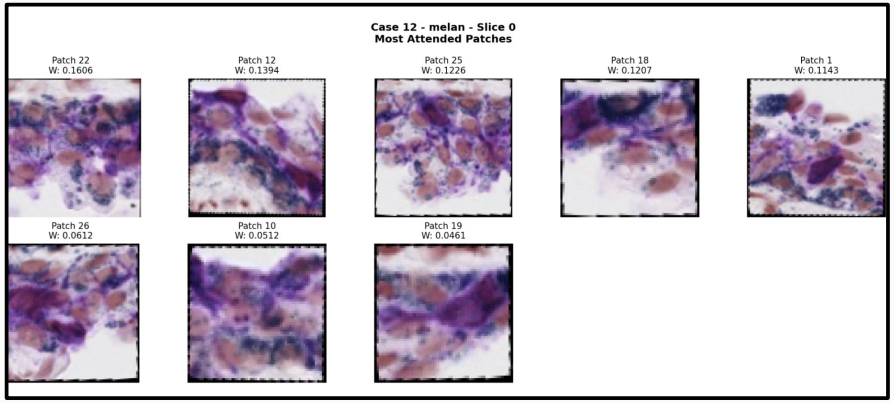


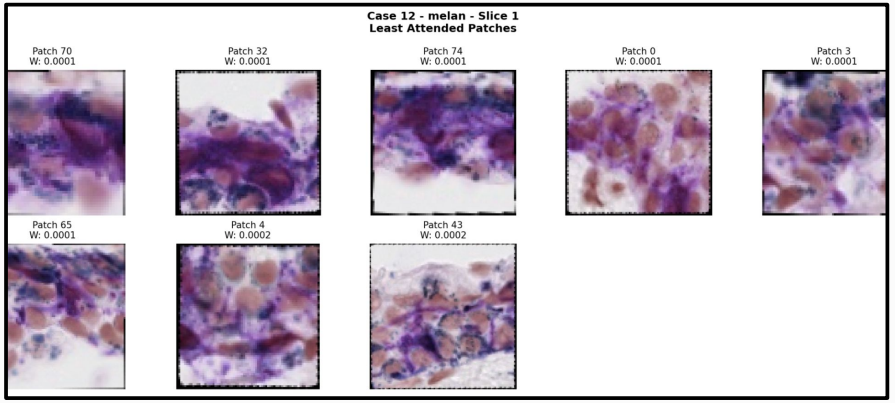


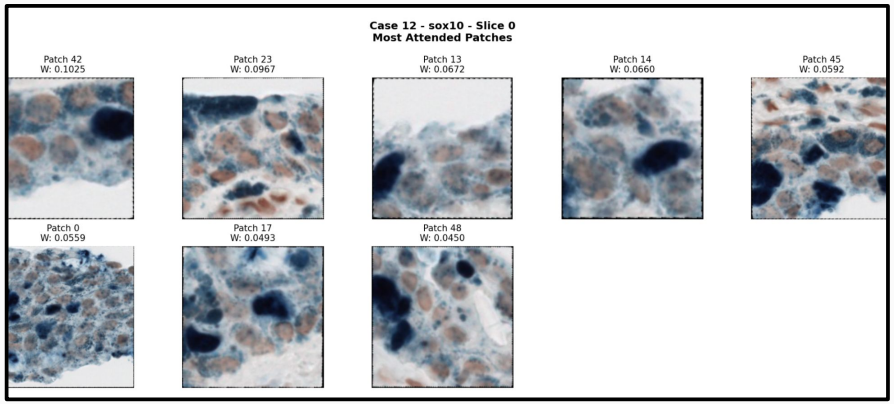


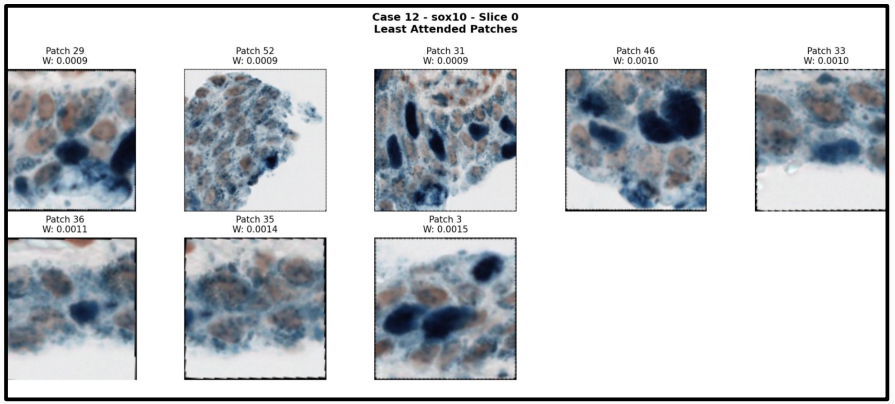
Taylor

Appendix: Full Stain & Slice Visualizations (Case 12)



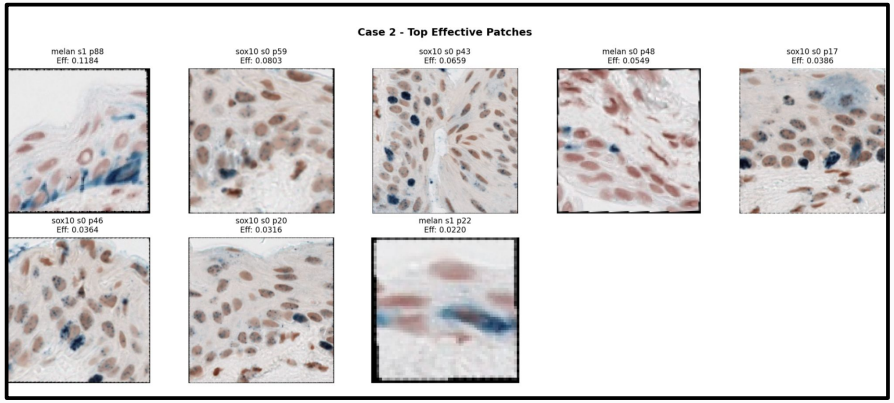


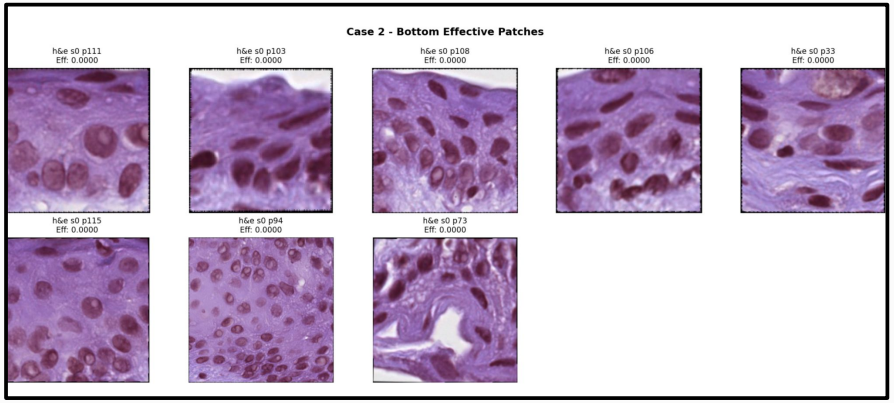


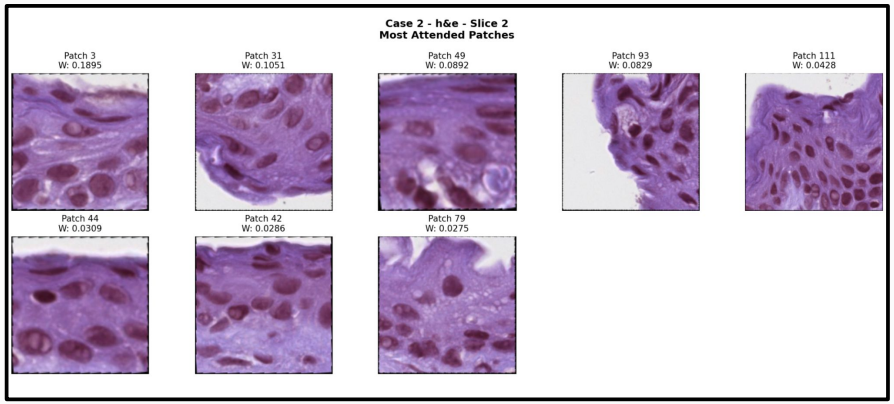


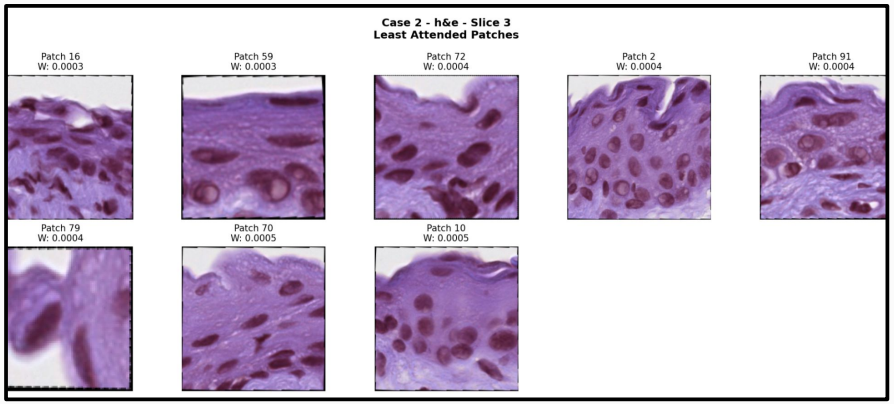
Taylor

Appendix: Full Stain & Slice Visualizations (Case 2)



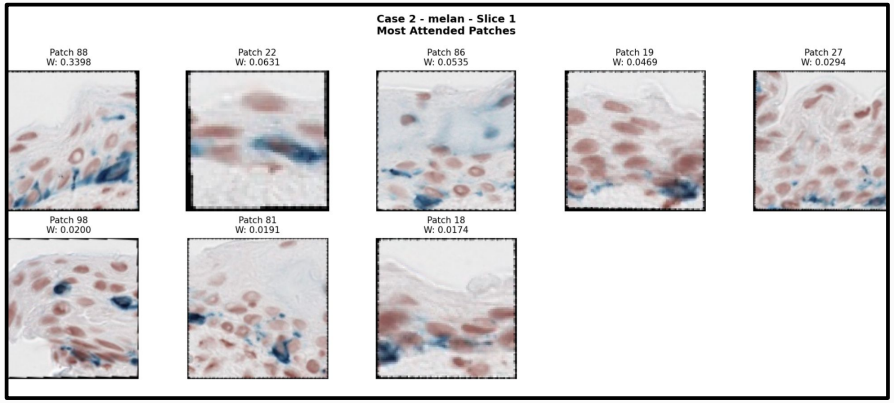


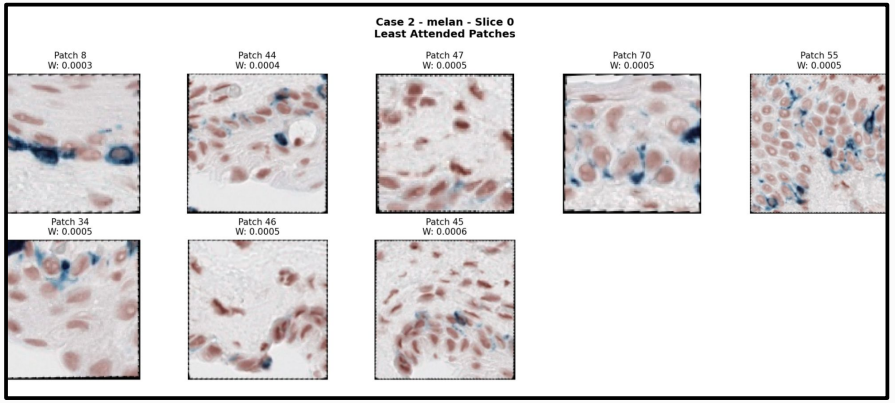


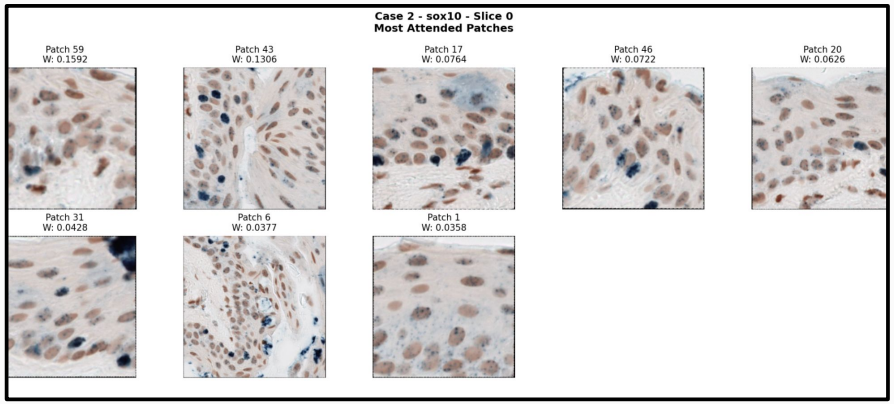


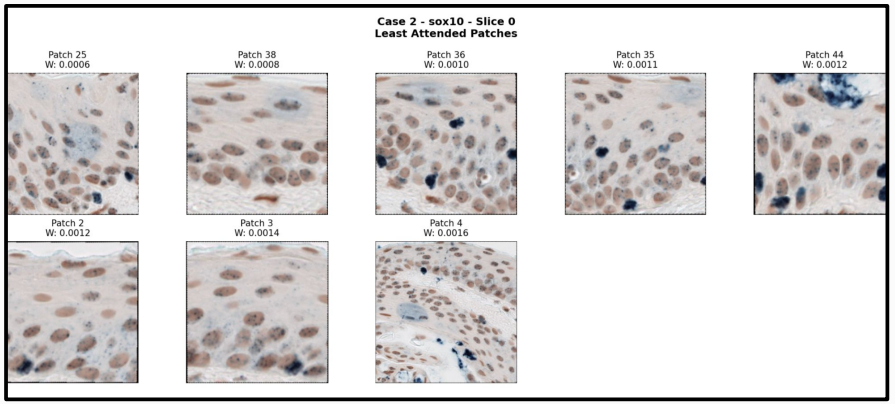
Taylor

Appendix: Full Stain & Slice Visualizations (Case 2)



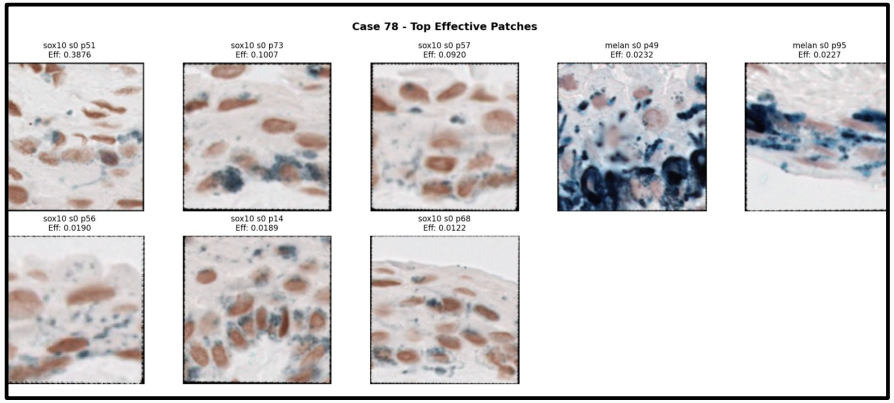


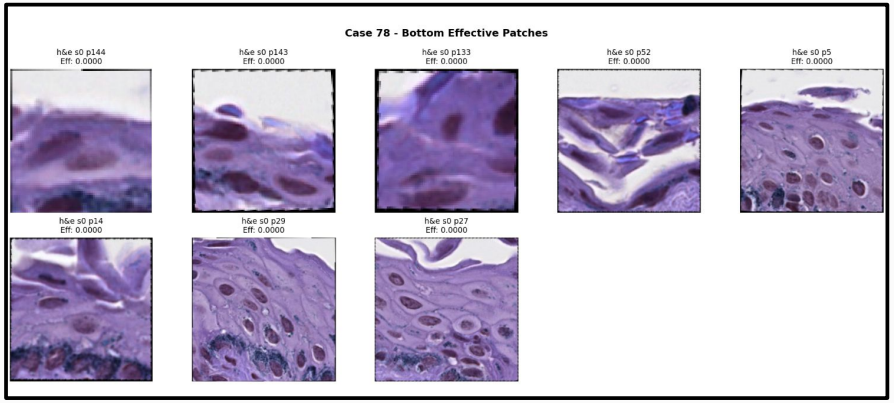


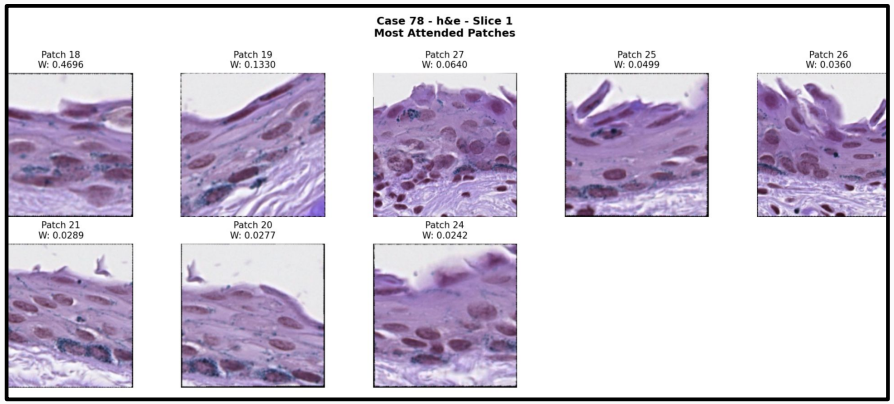


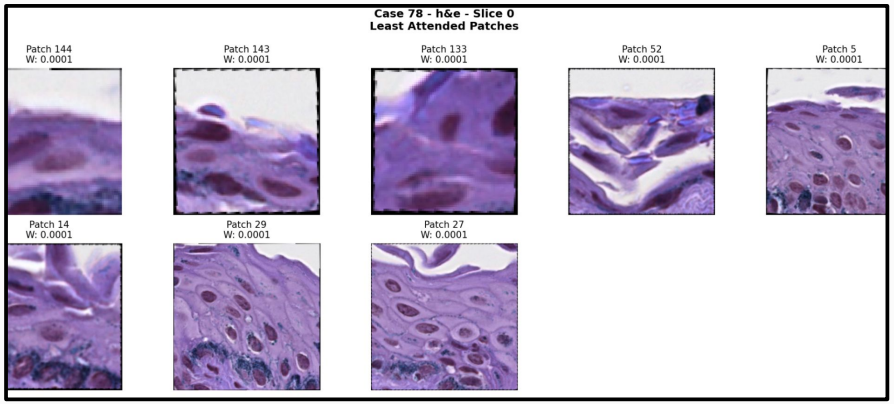
Taylor

Appendix: Full Stain & Slice Visualizations (Case 78)



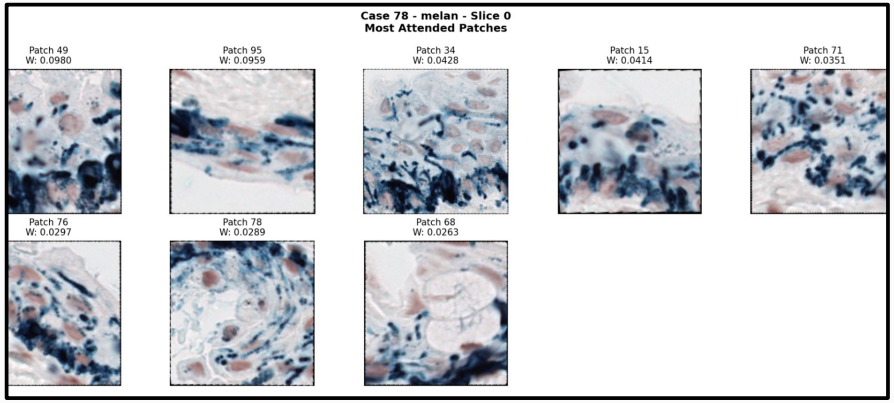


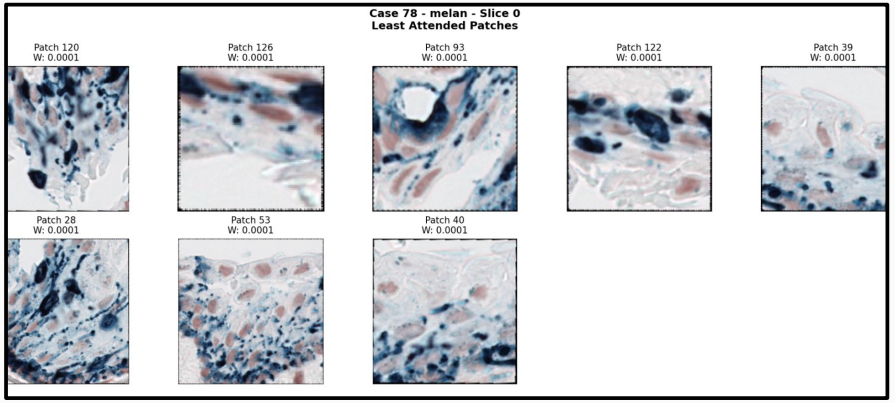


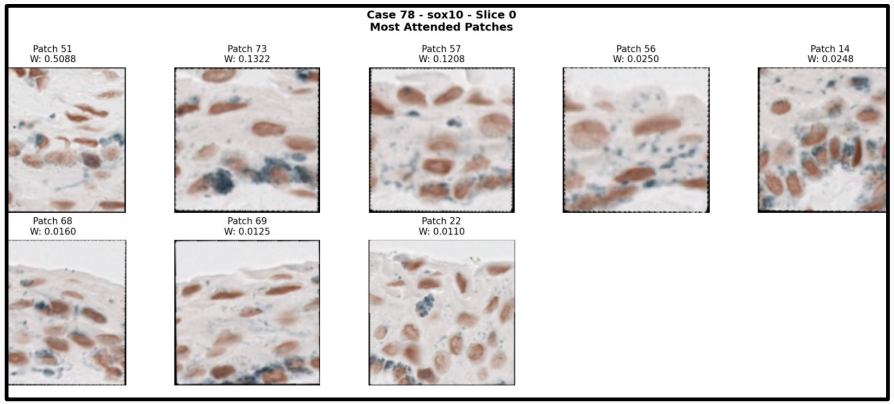


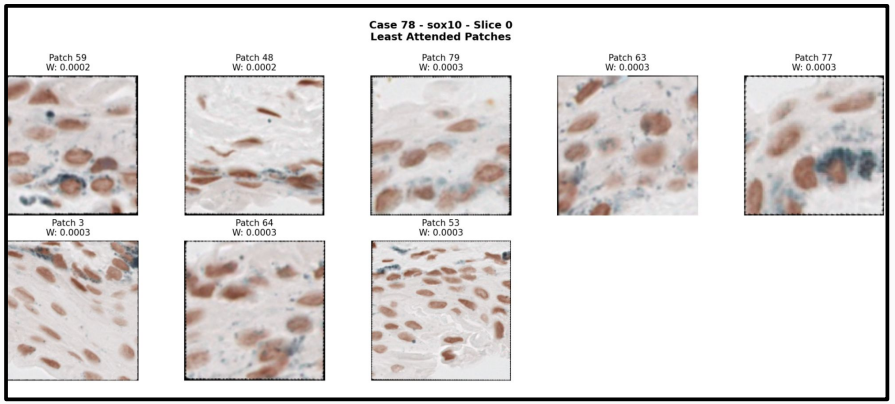
Taylor

Appendix: Full Stain & Slice Visualizations (Case 78)



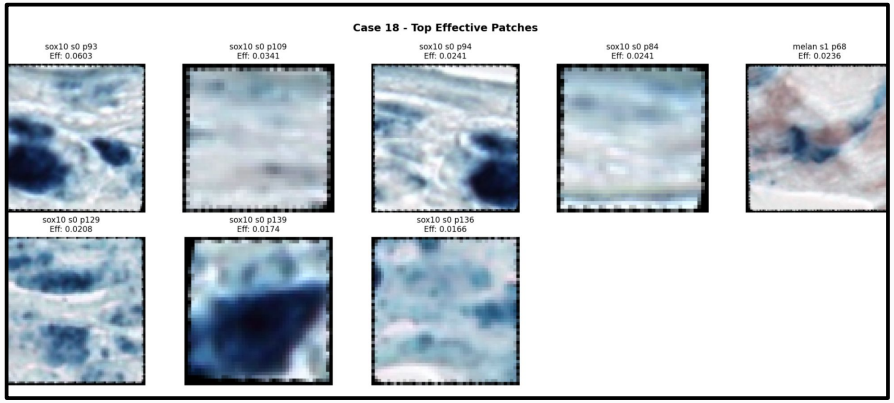


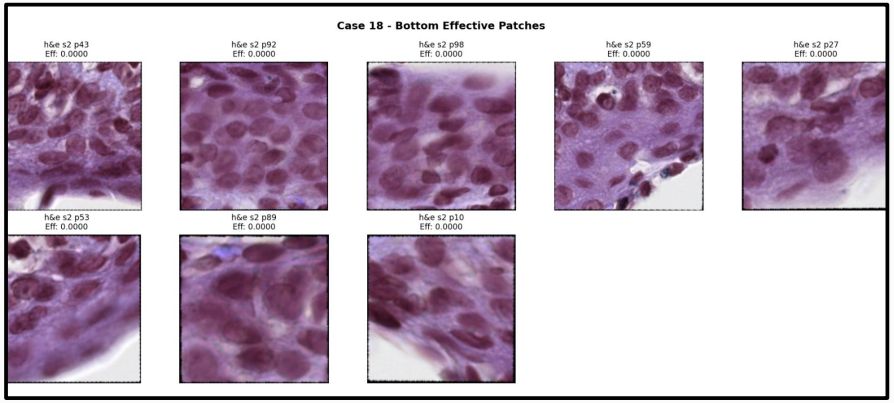


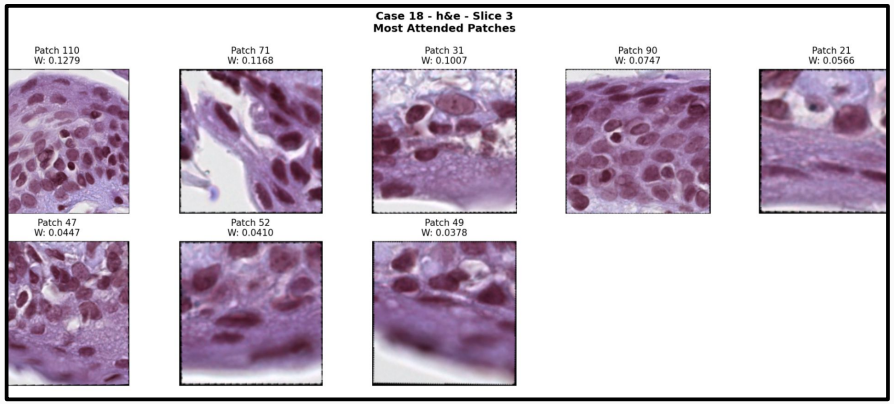


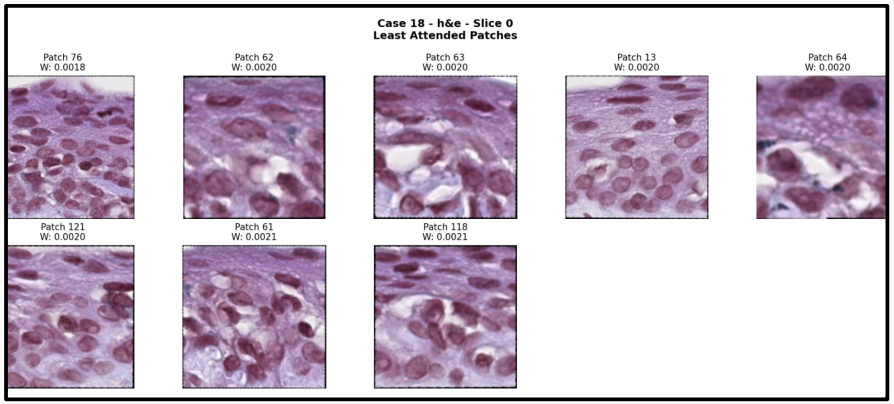
Taylor

Appendix: Full Stain & Slice Visualizations (Case 18)



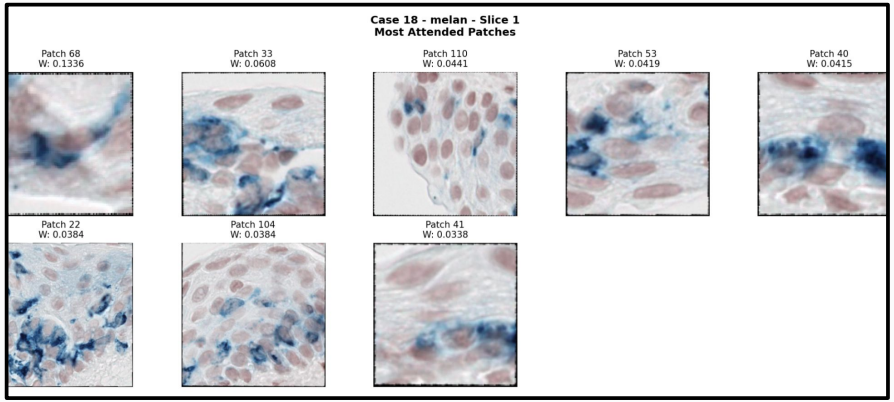


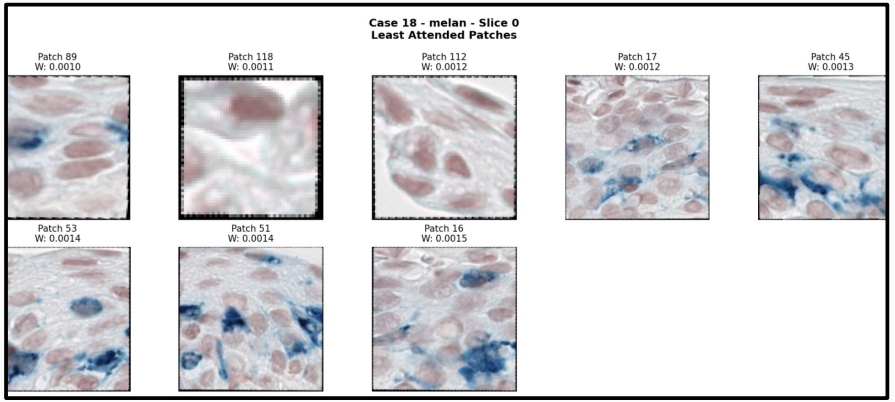


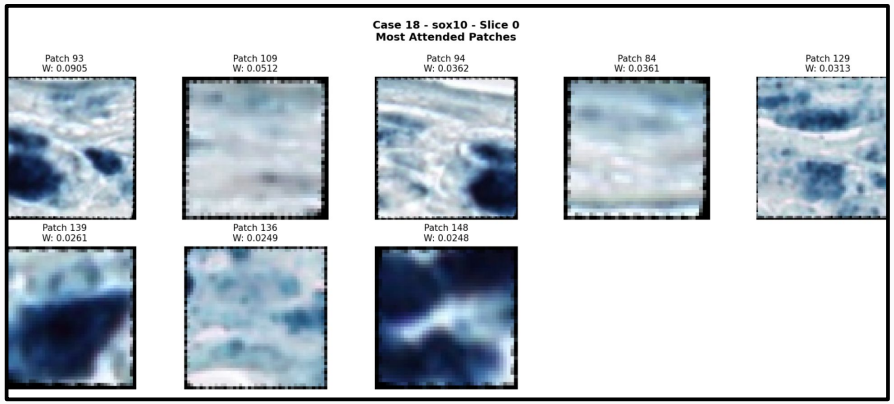


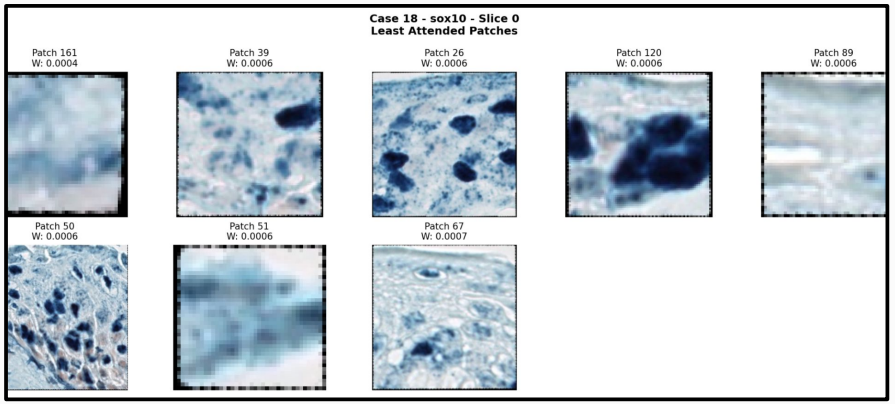
Taylor

Appendix: Full Stain & Slice Visualizations (Case 18)









Taylor